



INDIVIDUAL ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT073-3-2-CSLLT

Computer System Low-Level Techniques

HAND OUT DATE: 18 April 2026

HAND IN DATE: 15 May 2026

WEIGHTAGE: 60%

Name	TP Number	Intake Code
Mohib Bin Aamir	TP085584	APD2F2509CS(DA)

Table of Contents

1.0 Introduction to Assembly Language.....	
1.1 Overview of Assembly Language.....	
1.2 Importance of Assembly Language.....	
1.3 Purpose of the Project.....	
2.0 Research and Analysis.....	
2.1 Use of Low-Level Language in Cybersecurity.....	
2.2 Assembly Language in Malware Analysis.....	
2.3 Assembly Language in Reverse Engineering.....	
2.4 Assembly Language in Digital Forensics.....	
2.5 Summary of Research Analysis.....	
3.0 System Design.....	
3.1 System Overview.....	
3.2 System Functionalities.....	
3.3 System Assumptions.....	
3.4 System Flowchart.....	
3.5 Flowchart Explanation.....	
4.0 User Manual.....	
4.1 Tools Required.....	
4.2 Installation and Setup.....	
4.3 Steps to Assemble, Link, and Run the Program.....	

5.0 System Screenshots.....	
5.1 Source Code in Nano Editor.....	
5.2 Main Menu Output.....	
5.3 Ordering Items.....	
5.4 Final Bill Output.....	
6.0 Source Code Explanation.....	
6.1 Data Section.....	
6.2 BSS Section.....	
6.3 Text Section.....	
6.4 Arithmetic Operations.....	
6.5 Loops for Iterative Processes.....	
6.6 Jumps for Conditional and Unconditional Branching.....	
6.7 Input and Output Handling.....	
6.8 Discount and Tax Calculation.....	
7.0 Conclusion.....	
8.0 References.....	

1.0 Introduction to Assembly Language

1.1 Overview of Assembly Language

A low-level programming language known as assembly language allows programmers to communicate immediately with computer hardware. Assembly language provides more direct control over the processor, memory, registers, as well as system-level functions than high-level programming languages such as Java, Python, or C#. It defines machine-level actions with simple instructions referred to as mnemonics, for example MOV, ADD, SUB, MUL, DIV, CMP, and JMP.

In comparison to high-level programming languages, assembly language is rather closer to machine code. This indicates that each instruction generally ends with a simple and defined task. One instruction may for example send information into a register whereas other may multiply or comparison two values. Due to this, programming in assembly language requires extensive planning and a solid grasp of data flow through the computer system.

An assembler for the intel x86 architecture is known as Netwide Assembler or NASM. It is used for translating source code written in assembly language into object code which can be connected into an executable application. This assignment requires the usage of NASM and is not permitted to use TASM or MASM; however, NASM is acceptable. Based on the official NASM documentation, NASM is an intel x86 architecture assembler as well as disassembler.

1.2 Importance of Assembly Language

Since it allows programmers to understand the basic functions of computer systems, assembly language is important. It demonstrates how the CPU stores, moves, compares as well as processes data. Assembly language is still beneficial during circumstances demanding hardware control, speed, memory efficiency, and detailed program analysis, although high-level programming languages are easy to use.

As assembly language emphasizes the connection between software and hardware, it is often covered in computer systems and architecture modules. Registers, memory, addresses, stack operations, processor commands, as well as operating system interfaces. All of them are more easily understood via students because of it. These concepts are useful in areas such as digital forensics, operating systems, malware analysis, reverse engineering, and embedded systems.

In this project, a basic Point of Sale Billing System is built as an assembly language. The system demonstrates how low-level instructions could be used to build a basic business application. The key programming components required for the assignment are such as arithmetic operations, loops, and jumps.

1.3 Purpose of the Project

The aim of this project is to use NASM x86 assembly language to design and build a simple point-of-sale system. The Burger King POS Billing System is the name of the system built for this project. Even though this is not an official app for Burger King; instead, it is an academic prototype based upon a fast-food restaurant menu.

The user can select items from a menu, enter quantities, calculate the subtotal, apply a discount, calculate tax, and display the complete bill. Whopper burgers, cheeseburgers, long chicken, fish burgers, French fries, onion rings, coke, and sundaes are a part of the menu options. Burger King Malaysia's menu categories including Whopper, beef, chicken, fish, sides, dessert as well as beverages, are served as the model for these item categories.

This project illustrates the use of:

- 1) Calculations using arithmetic for price, subtotal, discount, tax, and final total
- 2) Loops for processing user input while displaying the menu very often
- 3) Conditional and Unconditional Branching jumps
- 4) The Linux system needs screen output and keyboard input

5) NASM assembler to assemble the source code

2.0 Research and Analysis

2.1 Use of Low-Level Language in Cybersecurity

As many cybersecurity activities need an understanding of how software behaves once it has been built, low-level languages such as assembly language are still important. The original source code is generally hidden when a program is compiled from a high-level language. Only the executable file, memory dump, or machine instructions can be accessible to security analysts. Assembly language helps analysts in understanding the original commands carried out via the processor in such conditions.

Cybersecurity experts investigate how programs connect to memory, registers, files, networks, and operating system functions using low-level knowledge. This is significant when examining suspicious software, evaluating weaknesses, and understanding how attackers take advantage of system flaws. Program behavior which may not be evident from the user interface or usual program output which could be found through low-level analysis.

Furthermore, buffer overflows, shellcode, memory corruption, as well as exploitation might all be understood using assembly language. These are technical fields where memory locations, registers, and control flow could be easily controlled via an attacker. An assembly language programmer or analyst is more familiar with how these attacks operate and how to deal with them.

2.2 Assembly Language in Malware Analysis

One of the primary fields where assembly language is beneficial is malware analysis. Usually, malware is used as an executable file, and the analyst has no access to source code. Reverse engineering tools, which illustrate the program's machine code in assembly language, are often used via analysts to understand what the malware does.

Malware analysts could find out what the malware does via examining assembly instructions. For example, they can find out whether the malware encrypts data, tries to hide itself, produce files, changes registry values, links to a command-and-control site, or steals information. Reverse engineers use assembly language to study malware and understand native executable files, based on IBM's malware analysis training.

Since malware often tries to hide its intent, assembly language is useful in malware investigation. It may use obfuscation, encryption, packaging, or anti-debugging techniques. In some conditions, high-level analysis may not be adequate. Analysts could monitor the original program flow and understand the basic functions of malware via examining assembly instructions.

A malware analyst may search for odd jumps, encrypted strings, unusual Windows API calls, or repeat loops which unpack code into memory. The analyst may better understand what the malware is doing via understanding commands such as MOV, CMP, JMP, CALL, PUSH, and POP. As an outcome, knowing assembly language is still important for analyzing malware.

2.3 Assembly Language in Reverse Engineering

Studying a program to learn how it functions lacking access to its original source code is called reverse engineering. Cybersecurity, software debugging, vulnerability research as well as digital forensics all often use it. Reverse engineering often needs to study to be able to understand the logic of compiled programs as they are transformed into machine code.

Executable files could be disassembled, and the assembly instructions showed using reverse engineering tools like Ghidra, IDA Pro, x64dbg, and ollyDbg. Analysts could examine functions, control flow, memory use, and program behavior with the help of these tools. Yet the tools are not sufficient on their own. To decipher the instructions means, the analyst needs to be acquainted with assembly language.

When identifying program decisions, assembly language is especially important. CMP, JE, JNE, JG, JL, and JMP are examples of instructions which illustrate how a program chooses various execution paths. Reverse engineers could recognize a license check, such as seeing how the program jumps to an approved or rejected field after comparing a user-entered key with a value expected.

Security analysis could also gain advantages from reverse engineering. When a program crashes, the analyst can find out what went wrong via looking at the assembly instructions close to the crash site. This can help security teams in identifying potential threat risks and engineers in fixing software flaws.

2.4 Assembly Language in Digital Forensics

Digital proof gathering, examination, and analysis are a part of digital forensics. Forensic analysts might need to look at executable files, memory dumps, or questionable system activity in some investigations. Analysts could easily understand what occurred inside the system during an incident using low-level language.

Forensic analysts could find out whether a program carried out questionable activities via assembly language. An analyst may look at whether a suspicious executable attempted to access memory, run convert processes, change files or communicate across the network. Reading disassembled code and understanding how instructions communicate with the operating system might be important for this type of study.

Low-level knowledge might also be important for memory forensics. Malware might not leave a clear file on disk while it operates in memory. Code fragments, memory structures, and process behavior might require examination by analysts. Circumstances in which high-level information is not available, their knowledge of assembly language allows them to understand what the code is carrying out.

Further, by allowing analysts to investigate evidence more carefully, low-level language skills support digital forensics. They allow investigators to understand the internal operations of a suspicious application in accordance with what they seem to achieve.

2.5 Summary of Research Analysis

Although most modern programs are built using high-level languages, assembly language continues to remain important. It is important as it is in close range to the hardware and shows the exact commands which the CPU is carrying out. This low-level viewpoint is extremely useful in digital forensics, cybersecurity, malware analysis, and reverse engineering.

When source code is not available, malware analysts can easily understand malicious executables via assembly language. It helps analysts in examining hidden behavior, control flow, as well as program logic in reverse engineering. It helps investigators in the examination of suspicious files as well as memory data in digital forensics.

With everything considered, assembly language is still beneficial since it provides a detailed grasp of how software functions at the machine level. Due to this, it is helpful in technical areas where knowledge of internal behavior is needed.

3.0 System Design

3.1 System Overview

A text-based Point of Sale sample built with NASM x86 assembly language is called the Burger King POS Billing System. By typing a number, the user can select an item from the fast-food menu shown by the system. The customer enters the quantity after choosing an item. Following, the system multiplies the price of the item by the quantity to figure out the item total, which subsequently gets added to the subtotal.

To enable the user to add many items in a single order, the menu is repeatedly displayed after each order. The system presents the subtotal, calculates the discount, determines the tax, and displays the final amount when the user selects a checkout. To halt the program, the user can also choose the exit option.

Rather than decimal values, all prices are stored in cents. RM18.40, for example, is kept as 1840. Since assembly language works with integer arithmetic more effectively than decimal floating-point data, this method reduces the difficulty of the calculation.

3.2 System Functionalities

Functionality	Description
Display menu	Shows available food and drink items with prices.
Item selection	Allows the user to select an item using a menu number.
Quantity input	Allows the user to enter the quantity of the selected item.
Item total calculation	Multiplies item price by quantity.
Subtotal calculation	Adds each item total to the running subtotal.
Discount calculation	Applies 10% discount if subtotal is RM50.00 or above.
Tax calculation	Applies 6% tax after discount.
Final bill display	Shows subtotal, discount, tax, and final total.
Invalid input handling	Displays an error message if the menu choice is invalid.
Exit option	Allows the user to end the program.

3.3 System Assumptions

Throughout the system's development process, the following assumptions have been made are given below:

- i) An academic prototype of the system has been developed
- ii) This is not an official Burger King app
- iii) Only numerical data is required from the user
- iv) Cents are used to store all costs
- v) A database is not used via the system
- vi) Transaction history is not permanently saved via the system
- vii) The 10% discount is set
- viii) Only in the case if the subtotal is RM50.00 or above then the discount is applicable
- ix) The tax rate is set at 6%
- x) The program requires NASM to operate in a Linux or WSL environment

3.4 System Flowchart

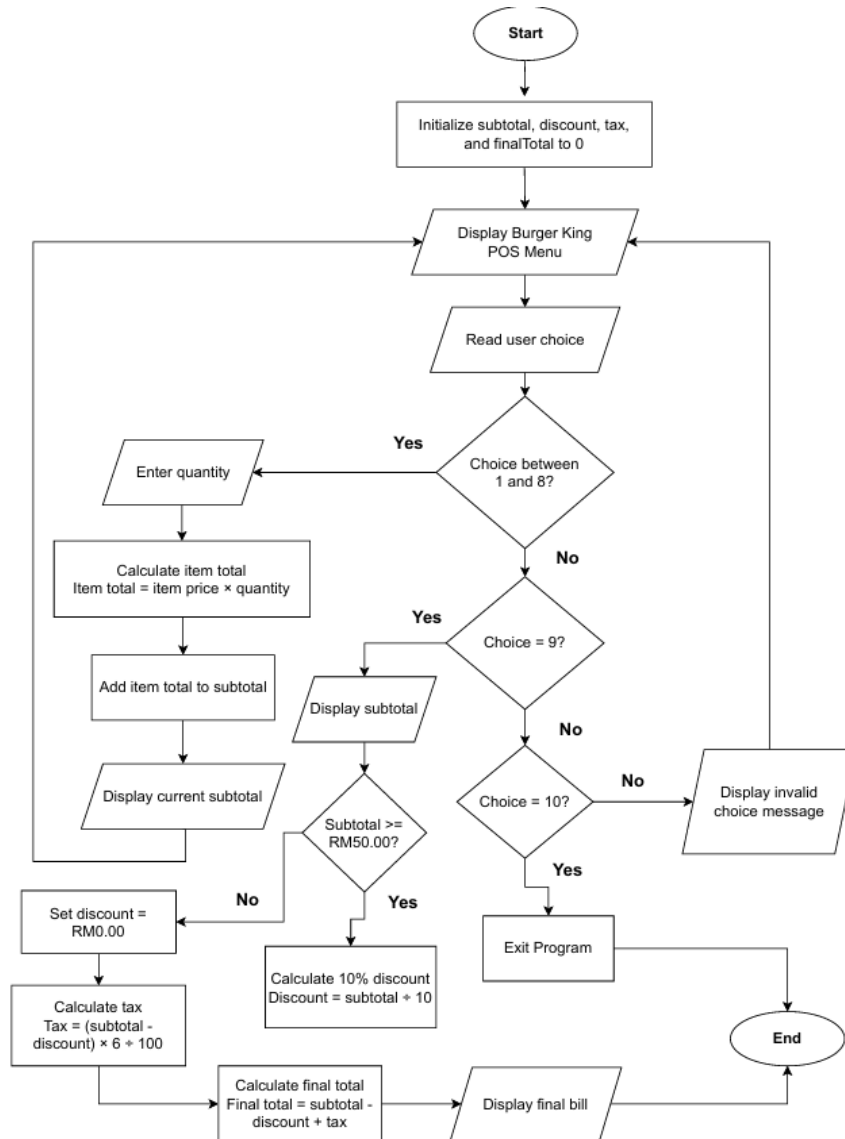


Fig 1: Flowchart of Burger King POS Billing System

3.5 Flowchart Explanation

The billing variables must be initialized to zero via the system. The subtotal, discount, tax, and final total represent a few of these variables. The Burger King POS menu displays when the

program is properly initialized. According to the item they would like to order, the user inputs a choice.

The user's input as well as the available menu items are compared via the computer. The system asks the quantity if the user selects an item from options 1 to option 8. The amount is then multiplied by the item cost, and the outcome is added to the subtotal. The user could then continue making orders when the system returns to the main menu.

The system proceeds to checkout if the user selects option 9. The subtotal appears first upon checkout. Afterwards, the software finds out if the subtotal is RM50.00 or more. A 10% discount is added if it is above RM50.00. The discount is set to zero if it is lower than RM50.00. The final sum is then displayed when the 6% tax is added.

The application finishes if the user selects option 10. The application goes back to the main menu and shows an error message if the user enters the wrong option.

4.0 User Manual

4.1 Tools Required

The following tools which are used are given below:

Tools	Purpose
Windows Subsystem for Linux / Ubuntu	Linux environment for running NASM assembly code.
NASM	Assembler used to convert <code>.asm</code> source code into object code.
Linker <code>ld</code>	Links object file into executable file.

Nano editor	Used to create and edit the assembly source code.
Terminal	Used to assemble, link, and run the program.

4.2 Installation and Setup

The windows subsystem for Linux is used to build and test the system in ubuntu. The user is required to install ubuntu and WSL first. The following commands must be used to update the system when ubuntu has been installed.

```
</> Bash
sudo apt update
sudo apt upgrade -y
```

NASM can be installed using.

```
</> Bash
sudo apt install nasm -y
```

The following command needs to be used to find out whether NASM was installed.

```
</> Bash
nasm -v
```

The terminal will display the installed NASM version if NASM is installed properly.

4.3 Steps to Assemble, Link, and Run the Program

The source code file name is:

```
</> Bash
pos.asm
```

To assemble the program, use:

```
</> Bash  
nasm -f elf32 pos.asm -o pos.o
```

This command turns the assembly source code into a 32-bit object file.

To link the object file, use:

```
</> Bash  
ld -m elf_i386 pos.o -o pos
```

This command creates the executable file named pos.

To run the program, use:

```
</> Bash  
./pos
```

5.0 System Screenshots

5.1 Source Code in Nano Editor

```

section .data
    titleMsg db 10, "==== BURGER KING POS BILLING SYSTEM ====", 10
    titleLen equ $ - titleMsg

    menuMsg db 10, "1. Whopper Burger          - RM18.40", 10
            db "2. Cheeseburger          - RM11.50", 10
            db "3. Long Chicken          - RM13.20", 10
            db "4. Fish Burger          - RM11.50", 10
            db "5. French Fries          - RM6.30", 10
            db "6. Onion Rings          - RM6.30", 10
            db "7. Coke          - RM4.00", 10
            db "8. Sundae          - RM5.50", 10
            db "9. Checkout", 10
            db "10. Exit", 10
            db "Enter your choice: "
    menuLen equ $ - menuMsg

```

Fig 2: Burger King POS Billing System in Nano editor

5.2 Main Menu Output

```

==== BURGER KING POS BILLING SYSTEM ====

1. Whopper Burger          - RM18.40
2. Cheeseburger            - RM11.50
3. Long Chicken            - RM13.20
4. Fish Burger             - RM11.50
5. French Fries            - RM6.30
6. Onion Rings             - RM6.30
7. Coke                    - RM4.00
8. Sundae                  - RM5.50
9. Checkout
10. Exit
Enter your choice: |

```

Fig 3: The Burger King POS Billing System main menu

5.3 Ordering Items

```
Enter your choice: 1
Enter quantity: 2
Item added. Current subtotal: RM36.80
```

Fig 4: User selecting menu items and entering quantities

5.4 Final Bill Output

```
===== FINAL BILL =====
Subtotal: RM141.30
Discount: RM14.13
Tax 6%: RM7.63
Final Total: RM134.80
Thank you for using Burger King POS!
```

Fig 5: Final bill generated by the Burger King POS Billing System

6.0 Source Code Explanation

6.1 Data Section

The software uses fixed data that is stored in the .data section. Messages, menu text, labels, as well as formatting characters come under this category. The .data section includes the program title, menu options, quantity message, incorrect choice message, checkout message and overall bill labels.

```
titleMsg db 10, "===== BURGER KING POS BILLING SYSTEM =====", 10
titleLen equ $ - titleMsg
```

Fig 6: Program Title Declaration in Data Section

Data bytes have been defined by using the db directive. The title message is stored in the titleMsg label. The message's length is stored in the titleLen value. The Linux sys_write system calls to print the message, this length is needed.

The menu is kept in the .data section also:

```
menuMsg db 10, "1. Whopper Burger      - RM18.40", 10
         db "2. Cheeseburger          - RM11.50", 10
         db "3. Long Chicken          - RM13.20", 10
```

Fig 7: Menu Items Declaration in Data Section

The program enables to demonstrate the items which are available along with their prices.

6.2 BSS Section

Variables with no set values at the first of the program are kept in the .bss section. Throughout the program execution, these variables are used.

```
inputBuffer resb 16
numBuffer resb 12

subtotal resd 1
discount resd 1
tax resd 1
finalTotal resd 1
```

Fig 8: Variable Declarations in BSS Section

The user's keyboard input is kept in the inputBuffer. While turning numbers into printable characters, the numBuffer is being used. Billing calculations benefit from the subtotal, discount, tax, and finalTotal variables.

6.3 Text Section

The program's executable instructions are placed in the .text section. The key program logic, menu control, item ordering sections, checkout calculation input and output processing, and then exit instructions all of them are included.

The program starts at the `_start` label:

```
_start:
    mov dword [subtotal], 0
    mov dword [discount], 0
    mov dword [tax], 0
    mov dword [finalTotal], 0
```

Fig 9: Initialization of Billing Variables

6.4 Arithmetic Operations

The system calculates item totals, subtotals, discounts, taxes, and final totals by using arithmetic operations. The program asks for the quantity when the user selects an item. Upon calling the `read_int` function, the quantity is saved in the EAX register.

```
mov ebx, 1840
mul ebx
add [subtotal], eax
```

Fig 10: Item Total and Subtotal Calculation

The amount is then multiplied by the item price in the MUL instruction. The result is then kept in EAX. The ADD instruction is then used to add the outcome to the subtotal.

```
xor edx, edx
mov ebx, 10
div ebx
mov [discount], eax
```

Fig 11: Discount Calculation Using Division

This divides the subtotal by 10 to calculate a 10% discount.

```
mov ebx, 6
mul ebx

xor edx, edx
mov ebx, 100
div ebx

mov [tax], eax
```

Fig 12: Tax Calculation Using Multiplication and Division

The amount after discount is then multiplied by 6 and divided by 100 to find out the 6% tax.

The final total is calculated as:

```
mov eax, [subtotal]
sub eax, [discount]
add eax, [tax]
mov [finalTotal], eax
```

Fig 13: Final Total Calculation

6.5 Loops for Iterative Processes

Loops are used via the system to process input and repeat the menu. The `main_menu` label is where the main menu loop begins.

```
main_menu:
    PRINT titleMsg, titleLen
    PRINT menuMsg, menuLen

    call read_int
```

Fig 14: Main Menu Display and User Input

```
jmp main_menu
```

Fig 15: Loop Back to Main Menu Using JMP

The user can order multiple items before checking out. The program will terminate after selecting only one item if this loop is not in place.

The `read_int` function makes use of different loops. Each character which the user enters is read via this loop, which then turns it from ASCII format into an integer.

```
parse_loop:
    mov bl, [esi]

    cmp bl, 10
    je parse_done
```

Fig 16: Input Parsing Loop

Until the Enter is identified, the loop keeps going. This allows option 10 and different single-digit and two-digit inputs to be handled via the program.

6.6 Jumps for Conditional and Unconditional Branching

In this program, jumps are essential as they control the execution flow. When the user's input and menu options have been compared, conditional jumps are used.

```
cmp eax, 1
je order_whopper
```

Fig 17: Conditional Jump for Whopper Selection

This indicates that the program proceeds to the order_whopper section if the user inputs 1.

```
cmp eax, 9
je checkout

cmp eax, 10
je exit_program
```

Fig 18: Checkout and Exit Menu Branching

This implies that option 10 ends the program, while option 9 sends it to the checkout.

```
mov eax, [subtotal]
cmp eax, 5000
jl no_discount
```

Fig 19: Discount Eligibility Check

This determines if the subtotal is lower than RM50.00, or 50000 cents. The program changes to `no_discount` if the subtotal is lower than RM50.00

```
PRINT newline, 1
jmp main_menu
```

Fig 20: Unconditional Jump Back to Main Menu

When an item is added, this always goes back to the program to the main menu.

6.7 Input and Output Handling

```
%macro PRINT 2
    mov eax, 4
    mov ebx, 1
    mov ecx, %1
    mov edx, %2
    int 0x80
%endmacro
```

Fig 21: PRINT Macro for Output Display

The Linux `sys_write` system call has been used via the PRINT macro to display output on the screen. `EAX = 4` describes the write system call in this process, and `EBX = 1` describes standard output, or terminal screen. The address of the message which needs to be printed is kept in `ECX` and its length is kept in `EDX`. Lastly, the Linux kernel is called using `int 0x80` to display the message.

```
read_int:
    mov eax, 3
    mov ebx, 0
    mov ecx, inputBuffer
    mov edx, 16
    int 0x80
```

Fig 22: User Input Handling Using sys_read

The Linux `sys_read` system call is used by the `read_int` function to read keyboard input. `EBX = 0` describes standard input which is the keyboard, and `EAX = 3` describes the read system call in this process. The maximum number of bytes which can be read is kept in `EDX`, while the address of the input buffer where user input will be kept is stored in `ECX`. The Linux kernel is then called to carry out the input operation via the `int 0x80` instruction.

6.8 Discount and Tax Calculation

The final bill is calculated in the checkout section. The subtotal is displayed first. The software then finds out whether the subtotal is RM50.00 or more.

The algorithm finds a 10% discount via dividing the subtotal by 10 if it is RM50.00 or above. The discount is set to zero if the subtotal is lower than RM50.00.

When the discount is applied, the tax is calculated at 6%. The formula used is:

```
xor edx, edx
mov ebx, 10
div ebx
mov [discount], eax
jmp calculate_tax
```

Fig 23: Discount Calculation Using Division

The final total is then calculated using:

```
calculate_tax:
    PRINT discountMsg, discountLen
    mov eax, [discount]
    call print_money

    mov eax, [subtotal]
    sub eax, [discount]

    mov ebx, 6
    mul ebx

    xor edx, edx
    mov ebx, 100
    div ebx

    mov [tax], eax
```

Fig 24: Tax Calculation Process

The PRINT MACRO and print_money function have been used to show the discount amount. The computer then calculates the amount when the discount is deducted from the discount of the subtotal. The 6% tax is calculated by multiplying the sum by 6 and dividing the outcome by 100.

7.0 Conclusion

Thus, the Burger King POS billing System displays how NASM x86 assembly language might be used to build a simple business billing application. Users can select items, input quantities, calculate subtotals, apply discounts, calculate taxes, and see the overall bill using the system.

Since the project uses loops, jumps, and arithmetic operations, it fulfills the main assignment requirements. Item prices, subtotals, discounts, taxes, and overall, all of them are calculated by using arithmetic operations. Loops are used to process user input and show the menu again. Jumps have been used for program control, discount checking, menu selection, and checkout decision-making.

While building this project, now I have a better understanding of low-level programming. I acquired knowledge regarding the way Linux system calls handle input and output, how registers have been used for calculation, how data is stored in memory, as well as how NASM turns assembly code into executable programs. Assembly language provides a better knowledge of how software communicates with computer hardware, yet more difficult than high-level programming languages.

8.0 References

GeeksforGeeks. (2025). *What is Assembly Language?* GeeksforGeeks.

<https://www.geeksforgeeks.org/computer-organization-architecture/what-is-assembly-language/>

Puckett, T. (2025). *Malware analysis and introduction to assembly language*.

<https://www.ibm.com/new/training/malware-analysis-and-introduction-to-assembly-language>

GeeksforGeeks. (2025). *What is Reverse Engineering Technique in Cybersecurity*

GeeksforGeeks.

<https://www.geeksforgeeks.org/computer-networks/what-is-reverse-engineering-technique-in-cybersecurity/>

Badman, A., & Forrest, A. (2025). *Digital forensics*.

<https://www.ibm.com/think/topics/digital-forensics>